© The British Computer Society 2021. All rights reserved.
For permissions, please e-mail: journals.permissions@oup.com
https://doi.org/10.1093/comjnl/bxab179

An Incremental Hierarchical Clustering Based System For Record Linkage In E-Commerce Domain

Furkan Gözükara [1,*] and Selma Ayşe Özel [2]

[1]*Department of Software Engineering, Faculty of Engineering, Toros University, Mersin, Yenişehir 33140, Turkey*
[2]*Department of Computer Engineering, Faculty of Engineering, Çukurova University, Adana, Balcalı Sarıçam 01330, Turkey*
**Corresponding author: furkan.gozukara@toros.edu.tr*

In this study, a novel record linkage system for E-commerce products is presented. Our system aims to cluster the same products that are crawled from different E-commerce websites into the same cluster. The proposed system achieves a very high success rate by combining both semi-supervised and unsupervised approaches. Unlike the previously proposed systems in the literature, neither a training set nor structured corpora are necessary. The core of the system is based on Hierarchical Agglomerative Clustering (HAC); however, the HAC algorithm is modified to be dynamic such that it can efficiently cluster a stream of incoming new data. Since the proposed system does not depend on any prior data, it can cluster new products. The system uses bag-of-words representation of the product titles, employs a single distance metric, exploits multiple domain-based attributes and does not depend on the characteristics of the natural language used in the product records. To our knowledge, there is no commonly used tool or technique to measure the quality of a clustering task. Therefore in this study, we use ELKI (Environment for Developing KDD-Applications Supported by Index-Structures), an open-source data mining software, for performance measurement of the clustering methods; and show how to use ELKI for this purpose. To evaluate our system, we collect our own dataset and make it publicly available to researchers who study E-commerce product clustering. Our proposed system achieves 96.25% F-Measure according to our experimental analysis. The other state-of-the-art clustering systems obtain the best 89.12% F-Measure.

Keywords: clustering; data mining; product clustering; record linkage; comparison shopping

Received 13 January 2021; Revised 23 August 2021; Editorial Decision 7 October 2021
Handling editor: Fionn Murtagh

1. INTRODUCTION

E-commerce is becoming an irrevocable part of our lives as each year passes, especially after the COVID-19 pandemic. With the emerging new technologies, advances in the Internet-based services and the increase in the customers' technological knowledge level, E-shopping is much widely used today than before. According to the US Census Bureau statistics [1], the US's electronic shopping volume had increased from 4.5% of total sales in 2011 to 16.1% in 2020. This massive increase in transaction volume attracts many businesses to start operating in the E-commerce area. Thus, many new E-commerce websites are starting to operate continuously. Due to the stiff competition among E-commerce businesses, many also discontinue their operation. Additionally, E-commerce websites regularly update products, prices and make discounts or campaigns because of the competition. Therefore, it may not be possible for average customers to keep track of changes in the E-commerce websites when making an online purchase. Comparison shopping agent (CSA) systems have been developed to provide the best prices for an online product and the other product features [2]. These systems collect the necessary information from different E-commerce websites and help their

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

customers to search for products with good prices. To our knowledge, the core of the CSA systems is the clustering mechanism because they need to crawl many different E-commerce websites to collect information about products and then identify the same products from different websites. A decent quality CSA must be able to cluster the same products from different E-commerce websites with at least 85% precision, according to Kannan *et al.* [3]. Thus, in our study, we propose a clustering system for CSA systems to cluster the same products crawled from different E-commerce websites.

Record linkage is the process of determining which records from two or more distinct data sources represent the same real-world entity [4]. Therefore, in this study, we define our clustering task as a record linkage problem. The contributions of this study are summarized as follows:

- An incremental/dynamic Hierarchical Agglomerative Clustering (HAC) system is proposed to solve the record-linkage problem of E-commerce products. To our best knowledge, such a clustering system has not been used before for this purpose.
- A novel product matching/filtering system that significantly improves the success of the clustering task is proposed.
- A new product code/identifier detection system for E-commerce products is developed.
- An evaluation approach for measuring clustering performance is presented by using an open-source clustering framework. As the evaluation of the clustering task may change due to the employed evaluation algorithm or how the algorithm is implemented, we believe that our proposed approach solves this problem and provides objectively comparable results.
- Very detailed experimental results of the state-of-the-art algorithms are presented.
- To our best knowledge, we provide the first publicly available E-commerce record-linkage dataset. Moreover, we also provide differently formatted versions of our dataset to make it possible for future studies to use the same data to evaluate their results and compare them with our results.
- The proposed system does not exploit the language-specific characteristics of the product names. Rather, language-independent characteristics of E-commerce products are utilized in the clustering process. Therefore, our system can be applied universally for all-natural languages.

The rest of the paper is organized as follows: in the next section, the related work for the record linkage is discussed. In the third section, details about the dataset used in this study are presented. The fourth section describes the proposed clustering system in detail. The experimental setup of the study is explained in the fifth section. The sixth section discusses the experimental results, which show the effectiveness of the proposed methods, and finally, the last section concludes the study.

2. RELATED WORK

Grouping the same entities from different sources into the same cluster is known as record linkage [4–7], deduplication [8, 9], name matching [10], data integration [11], data cleaning [12–14], duplicate detection [15–18], information integration [19], entity resolution [20–23], reference reconciliation [24], merge/purge problem [25], object identification [26, 27], entity matching [28], identity uncertainty [29], reference matching [30], hardening soft information sources [31] and robust reading [32] in the literature and has been widely researched. However, as can be seen from the different naming, there is no standard definition of this problem. Also, algorithms developed to solve this problem change for different domains such as E-commerce, health care, etc., because it requires domain-specific methodologies for obtaining high success rates [5, 8, 26, 33, 34]. Thus, techniques to be used in each application area for record linkage may vary. Furthermore, the majority of the previous works are done on a single category entity, for example only on restaurants, or only on authors, etc. However, in this study, we propose a system that makes record linkage for product names from an unlimited number of categories such as video cameras, led TVs, smartphones, hard disks, printers, fans, grass mowers, perfumes and such.

Several surveys have been published on record linkage problems recently, and they can be checked to have a clearer idea about the problem. Koudas *et al.* [35] have made a pervasive report on similarity metrics and algorithms used in clustering to solve record linkage problems. Winkler [36] has covered standardization, string comparators, parameter estimation, and link analysis topics for record linkage. Hall and Fienberg [37] have presented a survey of privacy-preserving record linkage methodologies and algorithms. Köpcke and Rahm [34] have reviewed frameworks used in the entity matching with considering blocking support. They have concluded that the recent frameworks are advancing toward systems that use training data and perform offline matching. Köpcke and Rahm [34] have also stated that it is impossible to compare the effectiveness and efficiency of the previously proposed systems due to the incompatibility in the used methodologies, measures, and test problems. Therefore, we need a method that can be used commonly by researchers for performance evaluation.

Aggarwal and Zhai [38] have published an extensive survey of text clustering algorithms. Christen [8] has published a detailed review of indexing techniques used in record linkage problems and has provided comprehensive experimental comparison results. Indexing techniques are used to reduce the number of candidate record matching. Thus, record-linkage performance is improved via fewer computations. Köpcke *et*

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

al. [39] have presented a comprehensive report evaluating more than 20 entity resolution approaches on real-world datasets from bibliographic and E-commerce domains.

Among the studies mentioned above, only Köpcke *et al.*'s study [39] on the E-commerce domain is similar to ours. However, there are several fundamental differences. Köpcke *et al.* [39] have used datasets only from two different data sources when experimenting, while we have experimented on a combination of 50 different data sources. They have not experimented with any incremental approach and chosen products only from manually determined electronic product categories. While the tested approaches can obtain very high F-measures on the bibliographic datasets, the same approaches have very low F-measures in the E-commerce domain. For example, while the combination of FEBRL [40] and FellegiSunter [7] obtains the best 97.4% F-measure on bibliographic datasets, its maximum F-measure for the E-commerce domain is only 53.8%, according to Köpcke *et al.*'s study. One significant deduction of their research is that while learning-based approaches can obtain higher success rates by using multiple attributes, they significantly increase the run time and become infeasible on large datasets. Additionally, it requires a very significant proportion of the dataset to be labeled for training to obtain decent results in the E-commerce domain.

In the following subsections, we summarize the related work in two subheadings so that the first subsection includes studies on the general record linkage problem, whereas the second subsection presents the studies on product matching, which is more relevant to our study.

2.1. Related work on general record linkage

Machine learning techniques are generally applied to solve the record linkage problem; as an example, Elfeky *et al.* [4] have presented a new tool for record linkage problems and three different machine learning methodologies that are a supervised method (i.e. classification), an unsupervised method (i.e. clustering), and a hybrid method that is a combination of both classifier and clustering methods. Experimental evaluations have shown that unsupervised and hybrid methods have limited accuracy. Tejada *et al.* [26] have proposed an object identification system that can automatically learn the appropriate weights of specific domains to minimize user input. They have utilized domain-specific features (e.g. Uniform Resource Locator (URL) of the company domain, phone of the restaurant domain, code of the airport domain, etc.). However, the proposed system is only applicable to a single category, such as only the restaurant category. Thus, the system is not suitable for a dataset that contains multiple categories such as restaurants, hospitals, airports, and schools simultaneously.

Sarawagi and Bhamidipaty [9] have proposed an interactive deduplication system that uses active learning. The system employs a trained classifier to classify records as duplicate or not. Their main contribution is to utilize active learning for picking up the most influential records to classify and obtain high accuracy with the minimized user input. Cohen and Richman [41] have presented techniques for entity name matching and entity name clustering tasks that are scalable and adaptive. They employ classification for their tasks and provide the desired clusters. Moreover, they do clustering or record linkage only on a single category of entities.

Grannis *et al.* [42] utilize the FellegiSunter probabilistic linkage method and the Expectation–Maximization (EM) algorithm to automatically determine the thresholds of the FS linkage method. For the identity uncertainty problem, Pasula *et al.* [29] have developed a model that utilizes the Markov Chain Monte Carlo (MCMC) algorithm and Bayesian networks. Since the MCMC algorithm is hardly scalable, distance metrics are also employed to fragment datasets into canopies. Hotho *et al.* [43] have proposed an ontology-based Text Documents Clustering Improvement System that utilizes the Bisecting K-means clustering algorithm for the clustering task and the WordNet to add the semantically higher concept words to the document vectors to improve the related documents' relevancy.

A hierarchical clustering system that utilizes frequent itemsets has been developed by Fung *et al.* [44]. Their method first generates initial clusters based on frequent global itemsets. These clusters are then divided into smaller clusters according to the inter-cluster similarity to generate final clusters by employing manually pre-defined thresholds, and final clusters are generated. Since their system primarily depends on the initial clustering methodology, it is static.

Jin *et al.* [5] have proposed a record linkage system that maps data pairs into high-dimensional Euclidean space by using the Euclidean distance between pairs as the first step. It employs a similarity join algorithm to determine which records are similar or not as the second step. For mapping into high-dimensional space, a slightly modified version of the FastMap algorithm [45] is used. For the similarity-joining operation, they have applied the algorithm proposed by Hjaltason and Samet [46].

An iterative record linkage system that utilizes linkage between similar objects has been proposed by Bhattacharya and Getoor [47]. The idea used in this study can be explained with the following example: if entity A and entity B are married to entity C, then A and B may be referring to the same person. Their motivation was author resolution and utilized co-authorship as a linkage criterion.

Cui *et al.* [48] have developed a hybrid clustering technique that applies both Particle Swarm Optimization (PSO) and K-Means clustering algorithms. Since the PSO algorithm is computationally costly, they use a hybrid model where (sub)optimal initial cluster seeds are found by using the PSO algorithm, and then, standard K-Means algorithm is employed to obtain better results by using these initially found cluster seeds. The success of the K-Means algorithm highly depends on the initial seed selection. Thus, this method improves the overall performance. Their method does not apply to our problem since they have

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

to provide the number of clusters, which is unpredictable in our case.

For efficient record linkage tasks, Yan *et al.* have proposed adaptive Sorted Neighborhood Methods (SNMs) [49]. They have primarily focused on the fixed sliding window size of the SNM method. An SNM sorts the dataset with respect to predetermined attributes. Thus, sorting-based methodologies are not suitable for our research since we aim to cluster different categories of entities, and each category can have very different attribute schemes. For example, the following two records are the same: 'PS3 DRAGON AGE INQUISITION', 'ARAL DRAGON AGE INQUISITION PS3 GAME'. Therefore, when a name attribute-based SNM is employed for blocking, they may never get on the same block.

Shen *et al.* [50] have developed a source-aware approach for entity matching problem that utilizes the properties of the sources or employs a strict and relaxed matching function. They assume that there is a strict matching function *S1* which has very high accuracy but limited recall, and another function, *S2*, can be defined based on more relaxed domain properties. They propose that higher recall can be obtained with minimized loss of precision by combining these two functions. Their system requires domain experts to define both *S1* and *S2* functions and decide either the domain is relaxable or not. They only have experimented on entity matching areas of mentions.

Inan *et al.* [51] have proposed a hybrid approach that combines cryptographic techniques and anonymization methods to provide a 3D tradeoff between privacy, cost, and accuracy. Their basic idea is using blocking on an anonymized dataset to reduce the cost of cryptographic methods. Their research focus is different from the focus of this study.

According to Hastie *et al.* [52], the dissimilarity measure choice affects the clustering task's performance more than the clustering algorithm's choice, and the choice of dissimilarity measure highly depends on the specific knowledge of the targeted domain. Even though this proposal is not correct for every domain in the clustering area, our experiments also verify that specific domain knowledge must be used to choose the correct algorithm and the dissimilarity measure. Thus, a single clustering system cannot be successful in every clustering domain. Because of this reason, we propose that a specific clustering system for each targeted domain needs to be developed. By this proposal, our study is unique in the literature because, to our knowledge, there is no identical targeted domain area study.

Kim and Lee [53] have proposed a record linkage blocking system based on an iterative Locality-Sensitive Hashing (LSH) technique that reduces record linkage run time compared with basic LSH and Multi-Probe LSH algorithms. They have called their system as HARRA (HAshed RecoRd linkAge).

A scalable entity matching framework for author referencing has been proposed by Rastogi *et al.* [54], who have split the dataset into subsets and run multiple instances of matchers on these subsets simultaneously. After each round, simple message passing and maximal message-passing systems are used to communicate between each matchers. The process continues to run matchers until all matchers are converged.

A system that utilizes product images to reinforce the text-based classification task of product matching problem has been developed by Kannan *et al.* [55], who proposed the Confusion Driven Probabilistic Fusion++ algorithm that improves the accuracy of the product matching task compared with the baseline text classifier.

Another entity matching system for large-scale RDF-based (Resource Description Framework) web data has been proposed by Böhm *et al.* [56]. They have composed an entity graph to maximize similarity using RDF format, reflexivity, symmetry, transitivity, and unique mapping per data source restrictions. By using the MapReduce-based assignment approach, they have achieved cluster-based scaling. Their system is highly different from our system because they heavily exploit the RDF format to calculate similarity and decide which entities are the same.

Gokhale *et al.* [57] have proposed a system that utilizes crowdsourcing and does not require any developer interaction to solve entity matching problem. The system uses crowdsourcing to label records, and by utilizing this labeling, both the blocking and the matching systems are trained by using Random Forest (RF) [52] algorithm. Finally, Zhang *et al.* [58] proposed a system that optimizes max-weight graph matching on multi-partite graphs for multi-source entity matching problems on enormous datasets. Their methodologies and aims are highly different from our study.

The methods mentioned above are not applicable to our problem since (i) most of them (e.g. [5, 9, 26, 29, 41, 42]) are suitable for single category entities, while our problem has infinitely many different product categories, (ii) some of them (e.g. [43]) are suitable for text clustering where documents are much longer than our product names, (iii) they (e.g. [44]) are developed for static datasets; however, we have a stream of data to be processed, (iv) they (e.g. [28, 47]) utilize well-formed attributes; however, product names do not have such attributes, (v) number of clusters are known in advance (e.g. [48]), but this is not applicable in our problem, or (vi) uses product images (e.g. [55]) which is not the case in our study. The below methods are more related to our study, and we explain their difference in the following paragraphs.

Bilenko *et al.* [49] have proposed an adaptive record linkage system that utilizes an online learning approach to identify the same products from different E-commerce websites. They have used the Averaged Perceptron (a space-efficient variation of the Voted Perceptron [59]) algorithm and multiple similarity functions for each identifying field, such as the price attribute of digital products or the author attribute of books. To obtain the training data, they have exploited the Universal Product Codes (UPCs) of the products. Only some products have this attribute, but the products that have UPCs are sufficient for the classification algorithms. First, the similarity functions are trained, and then, the HAC algorithm is used to cluster the

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

dataset. Even though their work looks similar to our research, there are several fundamental differences: (i) they exploit UPCs for supervised training, which is not available in our study; (ii) even though they mention that their method can be applied to a stream of data, they do not experiment with streaming data and do not explain details how can it be applied; (iii) they heavily use the structured data given in the comparison shopping site where they have obtained their dataset which is not possible in our research; (iv) even though they have experimented only on three different categories of products and have used all these leverages, they have obtained ∼40.0% F-measure, while we have achieved 96.25% for a much larger number of categories of products.

An entity matching system based on a generative model that utilizes hard and soft constraints has been proposed by Shen *et al.* [28]. The system's performance is highly affected by the dataset features, which are the core problem of entity matching systems. For example, in our study's domain, such features are either missing, erroneous or very hard to obtain. Thus, our proposed product code determination system is crucial for the product matching domain, which we explain in Section 3.

Bilenko *et al.* [6] have proposed an adaptive blocking framework to reduce pairwise comparisons. In our study, we propose domain-specific blocking methodologies that improve the success rate significantly.

Sahoo and Callan [60] have developed the first Incremental Hierarchical Clustering (IHC) of Text Documents. However, their system and approaches are much different from our study. They have experimented on news data, and our study and news data domain characteristics are entirely different. Additionally, they knew the number of categories, and they ordered the input by the date of the news. The order of the input data is critical when doing an IHC. Their tokenization and matching methods are also different from the methods in this study. As mentioned earlier, the record linkage problem requires highly domain-specific solutions.

2.2. Related work on product matching record linkage

There exist only a few studies that aim to make product matching. In this subsection, we make a detailed comparison of these studies with our study. One of these related studies belongs to Kannan *et al.* [3], who have proposed a system based on linking new product offers to an existing structured repository of products. In this method, at first, the new product offers are filtered by their category via classification. In the second phase, the new product offer is parsed and tagged with its features. After the parsing process, each of the new product attributes is weighted, and the possible candidate matching pool is reduced further by applying an attribute filter. Finally, their offline trained matching system assigns a similarity score to each one of the existing products in the candidate matching pool, and then, the new product is matched to the candidate that gets the highest score. This system uses the Microsoft Bing Shopping catalog, which has more than 5 million product offers. Firstly, having such a large and well-structured repository is not doable for every developer. Secondly, relying on such a repository requires this repository to be up-to-date. However, in our system, we do not have such a repository, and the definition of the attributes is not used as heavily as they do. Moreover, attributes may be missing too. Additionally, we do not require such a training set and well-structured categories to filter. Thus, we propose that our system is more applicable to a broader set of categories.

Köpcke *et al.* [61] have proposed a system for better product matching, which utilizes product code determination, product category filtering and manufacturer dictionary. However, these prerequisites are difficult to obtain in real-life scenarios. They have manually cleaned the product categories and the manufacturers' dictionary, which is a difficult process when hundreds of different E-commerce websites have thousands of different categories. As they have already obtained their data from a structured E-commerce product portal, manual data cleaning is possible. However, we use real E-commerce websites, and it is difficult when multiple different E-commerce websites are used. As an example, we list possible categories of 'iPhone Wallet Case' obtained from different E-commerce websites as follows:

- Cell Phones & Accessories > Cases > Wallet Cases.
- Cell Phones > Accessories > Cases & Protectors > iPhone Wallets.
- Apple Cases > iPhone 6/6 s Cases > Commuter Series Wallet Case.
- iPhone Accessories.
- Wallets, Cases & Skins.
- Cell Phones > Cell Phone Accessories > iPhone Accessories > iPhone Cases & Clips.
- Women > Accessories > Handbags & Wallets > All Handbags & Wallets > Phone Cases.

As can be seen from the above example, it is quite impossible to automatically clean these categories and make them refer to the same category. Additionally, their used category normalization and product category cleaning depend on batch clustering. When incremental clustering is being made, it may not work as well as batch clustering. Moreover, they also utilize a training dataset. The crucial part of training dataset utilization in the product matching task is that a significant percent (at least 25% according to [39]) of the entire dataset needs to be provided as the training dataset. When there are hundreds of thousands of products, it is not feasible to obtain that amount of training data for that many products. Their code extraction system also mainly depends on search engine querying, which is heavily limited by all major search engines. They have used Support Vector Machine (SVM) as a classifier and exploited category-based matcher training, which significantly favors them. Even though they have used manually fixed product category assignment and exploited the code extraction system of the search

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

engine, they have obtained the best ∼66% overall F-measure. When they cluster only mobile phones, they obtain their best F-measure, which is 89%. However, as we have mentioned, obtaining a clean manufacturer dictionary, directory of each product record, and product code extraction via search engines are not practical. Also, in many products, manufacturer names or product codes do not exist as in the product name '*2 Wire Universal Central locking Motor*'. We obtain a 96.25% macro-average F-measure with minimal human intervention on our entire dataset without such strict dependencies. Additionally, our algorithm is incremental, which is a necessity in the record linkage of products.

Petrovski *et al.* [62] have proposed a system that utilizes product feature extraction to achieve higher accuracy in product matching. Their system mainly consists of feature extraction methodologies based on manually defined rules. Moreover, they utilize an automatic linkage between free text properties and structured product properties to obtain features. Using a combination of training on the structured data, manually defined regular expressions and the dictionary search, they have obtained brand, model, storage, display, processor and dimensions properties of the products. Using all these properties of the products, they have employed Silk Link Discovery Framework [63] based on genetic programming approach and have obtained 82.4% F-measure at best. However, even though their domain is similar to ours, there are fundamental differences. First of all, they only have experimented with the digital products category by employing the Naive Bayes (NB) classifier. With this high limitation, they have heavily exploited the unique characteristics of the specific category. In our case, we do not filter our domain to a specific category and exploit its characteristic features because most E-commerce sites (especially in our country) do not have such structured hierarchical categories for training. Moreover, they have heavily employed training and manually defined rules to obtain better feature extraction, which requires expensive human labor in a big dataset, especially when there are multiple categories. Furthermore, many different categories of products do not contain such product features to utilize. Thus, it may not be possible to obtain higher accuracies. Their published F-measure is also based on 2500 matching pairs and 2500 non-matching pairs rather than clustering a certain number of different products, such as in our case. We have clustered 250 different products from 1721 records. This matching versus non-matching classification method also simplifies the task when compared with our clustering task.

Primpeli and Bizer [64] have proposed a system to improve the robustness of active learning when doing entity matching on E-commerce products. With their proposed method that applies unsupervised matching of the record pairs as a first step, they have reduced the variation of the results of different runs by 10–16%. Thus, they increased the robustness of active learning.

Ristoski *et al.* [65] have exploited the Microdata markup format of products and used them as structured training data. In this study, an existing product matching approach has been improved by using neural language models and deep learning techniques. They have employed Conditional Random Field (CRF) to extract product attributes from textual descriptions, as well as Convolutional Neural Networks to produce embeddings of product images for solving the matching problem and categorizing products. Also, an existing product database has been enriched with product data from the Web. Exploiting Microdata information of products certainly provides leverage; however, that information is still missing from many of the E-commerce websites. Thus, it is still not feasible to apply globally. The product matching problem is treated as a binary classification task and experimented with NB, SVM, RF and k-Nearest Neighbors classifiers. They utilize paragraph2vec [66] for unsupervised text-based feature extraction. Their database consists of laptops, TVs and mobile phones, which usually have very rich attributes and descriptions. Their dataset is not publicly available. They have obtained their best results as 73.75, 84.96 and 83.27% F-measure for the laptop, television and mobile phone categories, respectively, by using CRF with text embedding + image embedding, and RF classifier.

Luo *et al.* [67] have proposed a system to solve the entity name recognition problem of cross-border E-commerce commodity titles. They have used bag-of-words (BOW) representation and Term Frequency∗Inverse Document Frequency (TF∗IDF) weighting scheme (WS) to generate feature vectors like in our research. Differently, they utilize the Word2Vec model to represent the semantic meanings of the words. Then finally, the long short-term memory (LSTM) network is used for entity name recognition of the commodities. They have obtained a 64.58% F-measure, 59.20% recall and 71.03% precision with their proposed method. However, we have to note that what they aimed to achieve was slightly different from our objective. Their aim is to label entities as their commodity type, such as toothbrushes, not as a specific brand of toothbrush product.

The closest study to our research has been done by Li *et al.* [68], which includes a table that compares the results of numerous algorithms on their dataset. They have utilized the title and attributes of products rather than just the title and price attribute of the products like our research. Utilizing all of the attributes of products rather than the only price certainly improves accuracy. However, obtaining structured attributes of products is challenging when the number of different E-commerce websites crawled increases. They have collected information only from two E-commerce websites, while we have collected from 50 different E-commerce websites. They have proposed a Product Matching Model, which consists of the product title matching and the product attributes matching modules. A bidirectional LSTM network is utilized to extract product title features, and a convolutional neural network is used to obtain product attribute features. For product attributes and their values, two interaction matrices are built. A special convolution operation is used to extract matching features from

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

the interaction matrix of attribute values, and a K-Max pooling operation is applied to extract the most powerful features from the interaction matrix of attribute names. Finally, the output of title matching and attribute matching modules are fused by multi-layer perceptron to generate final matching features. Once the final features are obtained, SVM, NB and Decision tree are used for classification, and Mart and LambdaMart are employed for the ranking. Classification is done to compare the title of products to determine whether they are in the same class (same entity) or not. The product titles and attributes are used as queries with the aim to rank identical products at the top. Their proposed system obtains 92.50% F-measure, 93.40% precision and 92.60% recall on classification scenario on their dataset; meanwhile, our proposed system obtains macro-average 96.25% F-measure, 99.23% precision and 93.47% recall by using lesser product attributes (we utilize only the price). Moreover, they did not publish their dataset publicly. A detailed comparison of the results of our system with the existing systems is presented in Section 5.

3. DATASET USED IN THIS STUDY

As we could not find any publicly available dataset to use in our study, we have collected our own dataset by crawling the top 50 E-commerce websites in our country. Our implemented focused E-commerce crawler visited the E-commerce websites and saved the title and price features of the products to develop our dataset. In this study, only the title and price features of the products are utilized because these two features always exist for a commercial product. If the price feature is not available, it means that the product is either out of stock or expired; thus, it should not be displayed to users of the CSA system. Therefore, such products can be safely ignored. The crawling process has gathered 763 886 product offers. From these offers, 250 different offers are randomly picked to compose the dataset. The products are selected from only one of the major E-commerce websites, which sells a broad range category of products. Thus, every product is ensured to be different. Because usually, an E-commerce website does not list the same product more than once. The only exception we have found is that some websites tend to list different colors of the same product multiple times. This issue is taken into consideration when manual labeling is done.

Next, the entire dataset is manually scanned to find which one of the other E-commerce websites also contains these products. Then, the set of the same products from the other websites are labeled as the same product set. Different colors of the same products are labeled as the same product except for the products in which color is essential such as toner cartridges. Therefore, in our experimental dataset, there are 1800 records (from 49 E-commerce websites) of 250 different products, and 1721 (from 48 E-commerce websites) of the records have the price attribute.

Additionally, we randomly select 100 subsets from our dataset and call this dataset as the macro-average dataset for doing a more fair evaluation. Each subset in the macro-average dataset is formed by selecting 1000 random records from 1721 records with a price feature, and the selected records are randomly sorted. We have published these datasets on a GitHub[1] repository with different formats, including the preprocessed and the used format in our research. To the best of our knowledge, this is the first-ever E-commerce products dataset published in the literature, which contains data from more than two sources and over 90 different categories of products. More details about the dataset can be found in the GitHub repository. We also have to note that the order of the input does matter when incremental HAC is made. Thus, a macro-average dataset is vital to evaluate the success of the algorithms correctly. Moreover, to ensure comparability between different algorithms and methods, we have used the same order of the records in all of the experiments.

4. THE PROPOSED CLUSTERING SYSTEM

In this study, we aim to develop a clustering system that takes product titles and their prices crawled from different E-commerce websites and determines which titles belong to the same product. We developed a crawler that collects product titles and prices in our previous study [69]. In this study, we only propose a clustering-based system that returns a set of clusters so that each cluster contains product titles that belong to the same product. We implemented our system by using C Sharp[2] (C#) programming language and .NET 4.5 framework.[3] For database operations, we employed the MS-SQL server 2014 database management system. All the proposed algorithms were implemented from scratch. Only for some experimentations, we used external libraries such as LibSVMsharp[4] library.

In this study, we propose an HAC-based system that makes product title matching. In a standard HAC algorithm, the closest cluster pairs are merged until the desired cluster structure is obtained [70]. However, the most controversial part of the HAC is that when the merging should need to be stopped. Numerous techniques have been proposed for this problem. However, there is no standard or widely accepted solution for this problem. One of the most commonly employed techniques is drawing a Cluster Dendrogram and manually deciding where to cut the clustering. Providing the number of desired clusters is one other technique that is usually employed. More detailed information about clustering techniques can be found in the survey published by Berkhin [71]. However, the approaches mentioned above and many other existing solutions require

1 https://github.com/FurkanGozukara/Record-Linkage
2 https://docs.microsoft.com/en-us/dotnet/csharp/
3 https://dotnet.microsoft.com/
4 https://github.com/ccerhan/LibSVMsharp

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

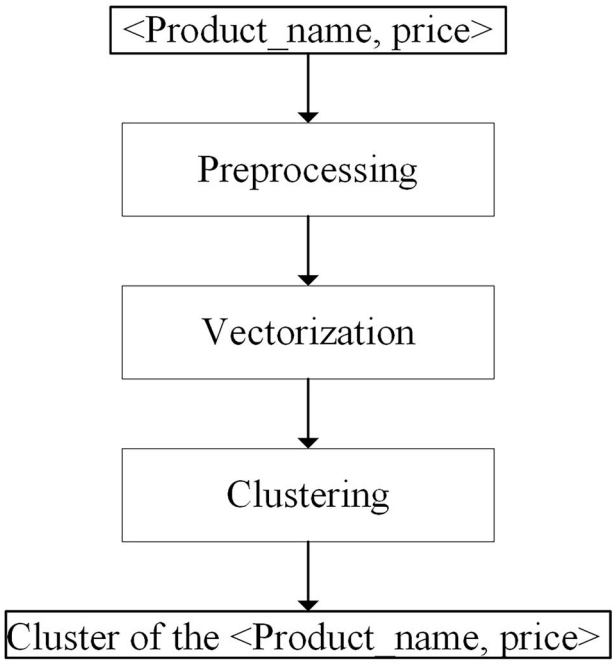


FIGURE 1. Main structure of our system.

manual analysis or the number of desired clusters. In our study, both approaches are not applicable as the number of clusters is unknown, and we have streaming data. Thus, we propose a new framework for doing Dynamic/Incremental HAC. We do not employ stopping criteria; instead, we use distance thresholding and multiple matching/blocking techniques until all records are processed.

The main structure of our system is presented in Fig. 1. Our proposed system consists of three stages: The product title and its price attribute are taken as input, then preprocessing steps are applied as the first step. After that, the product title and its price attribute are converted into a numeric vector which is then sent to our proposed clustering method as input to determine its cluster. The details of each step are explained in the below subsections.

4.1. Preprocessing

In the first stage, each product title and price attributes are preprocessed. In the preprocessing stage,

(i) Diacritics[5] are transformed into non-diacritic form.
(ii) Lower case conversion is applied.
(iii) Unique string replacements are applied.
(iv) Special semantic normalizations are applied.
(v) A novel word addition system is employed.
(vi) Punctuation marks are removed.
(vii) Non-ASCII characters are removed.
(viii) Words with less than 2 or greater than 30 characters are discarded.
(ix) Character-based N-gram transformation is applied.
(x) Finally, the title of the product is transformed into a BOW format.

In the BOW methodology [72], the text is split by space character into words, and each unique word represents a feature which is then assigned a numeric weight to represent the product record as a numeric vector of term weights. In the unique string replacement step, we do abbreviation normalization. For example, ″ is replaced with '*inc*'. In semantic normalization, we make synonymous word replacements. As an example of semantic normalization, '*refill*' is replaced with '*cartridge*'.

The particular word addition system is our novel proposal. After extensive experimentation and research, we have discovered that there are three special characters '-', '.' and ':' used in the product titles. We split the product title from these characters, and also we remove these characters from the title and use all obtained strings. For example, the word '*11-n000nt*' is converted into '*11*', '*n000nt*' and '*11n000nt*'.

We also apply character-based N-gram transformation to product titles. N-gram of a string is a set of substrings, each of which is a substring formed by N adjacent characters [73]. For example, a 2-gram representation of '*hospital*' contains '*ho*', '*os*', '*sp*', '*pi*', '*it*', '*ta*' and '*al*'. When employing character-based N-gram transformation, we have kept the original title as well. Therefore, our method uses the original title plus character-based N-gram transformed version.

The below example shows how preprocessing is applied to product title PC DRAGON AGE INQUISITION.

(i) **Raw Title**: PC DRAGON AGE INQUISITION
(ii) **Remove Diacritics**: PC DRAGON AGE INQUISITION
(iii) **Lowercase**: pc dragon age inquisition
(iv) **Unique string replacements**: pc dragon age inquisition
(v) **Special semantic normalizations**: pc dragon age inquisition
(vi) **Word addition**: pc dragon age inquisition
(vii) **Remove punctuation marks**: pc dragon age inquisition
(viii) **Remove non-ASCII chars**: pc dragon age inquisition
(ix) **Min-max word length**: pc dragon age inquisition
(x) **Original + 2 grams**: pc dragon age inquisition pc cd dr ra ag go on na ag ge ei in nq qu ui is si it ti io on
(xi) **BOW**: pc dragon age inquisition cd dr ra ag go on na ge ei in nq qu ui is si it ti io

4.2. Vectorization

After the product title is preprocessed, the frequency of each unique word in the product title is calculated and saved as the raw-frequency of the word. These frequencies are used to generate a numeric vector representation [74, 75] of each

[5] Diacritic—Wikipedia, https://en.wikipedia.org/wiki/Diacritic

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

product title where each element in the vector is the weight of the corresponding word, and generally, weight is the raw term frequency which is also known as TF value. Since many string-based algorithms work only on the numerical vector representation of strings, we have chosen this methodology. Additionally, it is possible to improve the success rate significantly by employing a suitable WS for words. Therefore, in this study, several WSs, including TF, IDF and TF∗IDF, are employed, and their performances are compared.

Although TF is generally computed as the number of occurrences of a term in a product title (or document), there exist several variations of the TF formula, including binary, raw frequency, log normalized, double log normalized and BM25 that are given in Section 5. IDF is another WS that gives higher weight to rarely occurring terms in the collection because they are more critical to discriminate a document from others. Several methods exist in Information Retrieval (IR) domain to compute IDF value for a term. In Section 5, formulas used to compute IDF value for a term are listed. We used all these variations of TF and IDF in our experimental evaluation. In general, using TF and IDF together to assign a numeric weight to a term gives better performance in IR studies. Therefore, we also used the well-known TF∗IDF weighting, which is computed as the product of the TF and IDF values of a term, and compared its performance in this study.

In addition to these, we still keep word representation of product titles as well for our specific methods. Once the numeric vector is ready for the product title, the third stage starts. The weighting methods that are used for vectorization in this study are presented in detail in Section 4.

4.3. Clustering

In the third stage, the system starts to cluster records one by one since our system does online (incremental) clustering. For the input vector, it can be either a new product or a newly emerged product cluster; the candidate cluster set is obtained by applying traditional blocking. If the compared single-product/product-cluster pair have at least 2 matching features with the input, the compared single-product/product-cluster is added to the candidate cluster set.

Once the candidate cluster set is obtained, the feature vector weighting function is called on the new input and the candidate cluster set. This local WS is the key to online clustering. Once weights of features are calculated, distances between cluster/record pairs are calculated with a distance metric. Therefore, how well each feature is weighted becomes very important, as stated by Tejada *et al.* [26]. We have experimented with many different WSs (e.g. TF, IDF, TF∗IDF) and distance metrics (e.g. Euclidean Distance) that are explained in Section 4.

To calculate the distance between cluster pairs, we have experimented with many distance functions. When calculating the distance, we do not merely choose the pair with the minimum distance like employed in the standard HAC. We employ different matching functions, and the power of our proposed system comes from these matching functions. As stated earlier, the success of the clustering algorithms depends on the application domain. Thus, we exploit some of the characteristics of E-commerce products. However, unlike the previous works, we only utilize the title and the price of the products.

The distance calculation process for each cluster pair is a costly operation. Because of this reason, our matching functions are employed before the distance calculation process, and if one of the matching functions fails, the operation is terminated, and the distance is not calculated. Early termination strategy (using matching functions) speeds up the process significantly and improves performance. The process is as follows: (i) distances between all of the candidate product (or cluster) pairs are calculated; (ii) the pair of products (or clusters) having the minimum distance are either merged into one cluster, or the product is put into a new cluster based on an empirically pre-defined distance threshold and (iii) if a new cluster emerges, this newly emerged cluster is checked against all existing clusters for a possible merge, and this re-clustering process continues until no new cluster emerges. The proposed clustering algorithm is given in Fig. 2.

Matching functions are employed to speed up the distance calculation and prevent miss-merges, thus improving the system's performance. We have four different matching functions as described below, and if the compared product pair passes all of the matching functions, their distance is calculated.

(i) The first matching function is based on the price feature of the products. If the price ratio between record pairs is over a pre-defined threshold, they cannot be the same product. For example, the below two products are very similar when only the title feature is considered, but their price ratio is 2.27, and we conclude that they are different products:

- Sony Xperia Z2 Android Phone: 1821 TL.
- Sony Xperia M2 Dual Android Phone: 799 TL.

This price-based matching system is also very efficient to prevent miss-clustering of the product and its accessories. For example, Köpcke *et al.* [61] have achieved very high success rates at the expensive products while failed to achieve such high success rates on the accessory products, because accessory products contain many features of the main products. Their success could have been much higher if they had employed a price ratio as we do. The price ratio check is swift and prevents many unnecessary expensive distance calculation processes of different products.

(ii) The second matching function we employ is supervised brand filtering. If the compared products contain different brands, they cannot be the same product. For this predicate, both products must have one of the pre-defined brands, and

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

Algorithm: Incremental Hierarchical Agglomerative Clustering
Input: Vector of product names, and their prices
Output: Clusters of the products

```
begin
    for each product vector p to be clustered do
        Generate candidate cluster set C for the input product vector p
        for each candidate cluster c_i in the set C do
            Check price_matching_rule for p and c_i
            Apply brand_filtering_rule for p and c_i
            Make word_conflict_check between p and c_i
            Apply product_code_matching between p and c_i
            if p and c_i passes all matching rules then
                Calculate distance d_i between p and c_i and store d_i
            end if
        end for
        Find the candidate cluster c_i that has the minimum distance d_i
        if d_i > max_distance_threshold or candidate cluster set is empty then
            Create a new cluster c_new and put p into this cluster
        else
            Insert p to cluster c_i
            Insert c_i to the beginning of to be clustered list and remove c_i from cluster list
        end if
    end for
end
```

FIGURE 2. The proposed clustering algorithm.

these brands have to be different. If only one of the compared products contains a brand or none have any brand, this filtering is not employed. For example, '*Kingston*' and '*Corsair*' are different brands, and the products that contain them cannot be the same product.

(iii) The third matching function is the word conflict check. There are certain strings that conflict, and if a conflicting word pair appears in the record pair, then these records cannot be the same products. Currently, domain experts generate this predefined list. For example, '*pc*' and '*ps3*' are conflicting words. One of them defines computer games, while the other one defines PlayStation games. An excellent example of such a case is shown as follows:

- **PC** DRAGON AGE INQUISITION.
- **PS3** DRAGON AGE INQUISITION.

The titles of the two given products are incredibly similar, and the only way to discriminate them is the word conflict system. Another case in which we use the word conflict filtering is the color names for printer cartridges. If the compared product titles contain the word 'cartridge' and different color codes, they are discarded even if they are very similar such as the case below:

- Canon CRG-731C Toner Cartridge Blue.
- Canon CRG-731 M Toner Cartridge Red.

(iv) The fourth matching function is based on product codes, usually given by brands to identify products uniquely [61]. Köpcke *et al.* [61] have heavily utilized this feature of products. However, our system works significantly differently and efficiently than their system. The biggest obstacle of their product code system is that it depends heavily on the public search engines. Today, none of the major search engines allows such automatic querying, and when there are thousands of products, their system becomes infeasible. The pseudocode of our product code matching algorithm is shown in Fig. 3. The algorithm aims to determine whether both of the compared records contain a genuine product code or not. If both records are determined to contain genuine product codes, these product codes are checked whether they are the same or not. If both contain genuine product codes and the codes are different, the system decides that they cannot be the same product even

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

if they get the closest distance among the candidate set. For example, the algorithm can successfully determine that the below products are different

- 18.5 PHILIPS **193V5LSB262** 5MS BLACK LED MONITOR.
- 19.5 PHILIPS **203V5LSB26** 5MS BLACK LED MONITOR.

The code detection system utilizes five different parameters and one pre-defined list, which are explained as follows:

(i) *minLength*: After conducting experiments, we have discovered that words with lengths that are less than or equal to three characters are usually not product codes. Therefore, for a word to be a product code, we define a minimum length threshold.

(ii) *specialCodeIgnoreFeaturesList*: This list is used to hold special product features, which are usually incorrectly identified as a product code. For example, 20HZ and 24INC are not product codes. This list holds such identifiers as 'hz' 'inc' to prevent false detection of product codes.

(iii) *minSubstringRatio*: If compared candidate products start with a certain number of the same characters, they are likely to be the same product. This parameter defines the minimum percentage of the same character count. If they do not start with a specific number of the same characters, the first stage of the algorithm is terminated. Then, the second stage of the algorithm starts, and the second stage evaluates whether they contain the same product code or not.

(iv) *editDistanceProductCode*: This parameter is used in two different places. First, it determines the maximum length difference of candidate codes. If the length of one of the candidate codes is much higher than that of the second one, they are discarded and evaluated at the second stage of the algorithm. This parameter also defines the maximum edit distance allowed between compared candidate product codes. If these candidates' edit distance is greater than the pre-defined threshold, they cannot be the same product code.

(v) *minLCSRatioWordLength*: This parameter is utilized in the second stage of the algorithm. If no genuine product codes are found in the first stage, the algorithm executes the second stage. This parameter defines the minimum length of the compared candidate product code pairs. If one of the candidates has a smaller number of characters than this parameter, they are not further evaluated.

(vi) *minLongestCommonRatio*: This parameter defines a minimum value for the percentage of how many characters need to be the same between the compared candidate product code pairs. The percentage is multiplied by the length of the smaller length having product code pairs.

5. EXPERIMENTAL SETUP

We have conducted many different parameter/metric/algorithm experiments because, in record-linkage, there is no single algorithm or methodology that performs best for all domains. Additionally, how the features are formed and weighted is extremely important in the distance computations between strings. The tokenization process assigns the best representative numerical weights to each feature in the vector form. Therefore, we have tested the following weighting metrics and vector normalizations to find out which one may perform best in the product matching domain:

Vector Normalization: We test the three options: None, L_1, L_2. The L_P normalization is presented in equation 1, where x_i is the ith element of the vector x, n is the number of elements in the vector and p is integer either 1 or 2

$$\|x\|_p := \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}. \quad (1)$$

WS: TF, IDF and TF∗IDF are used.

TF Variances: To compute TF value for a term Binary, Raw Frequency, Log Normalization, Double Normalization and best matching 25 (BM25) metrics are used and compared. Table 1 presents how each TF variant is computed.

IDF Variances: To compute IDF value for a term Unary, IDF, IDF Smooth, IDF Max and Probabilistic IDF methods are experimented. In Table 2, we summarize how each IDF variant is computed.

BOW: To create a BOW representation of each product title, 1-Word Gram, N-character Grams, 1-Word Gram + N-character Grams are compared.

An extensive study of the above metrics is presented by Paltoglou and Thelwall [76], where we have taken the TF formulas. Table 1 lists the equations that we have used for computing TF [76], where $f_{t,d}$ is the frequency of term t in document d, k_1 and b are constants of BM25, dl is document length and avg_dl is the average document length of all documents.

Variants of IDF weights [76] are shown in Table 2, where N is the number of documents, N_t is the number of documents that contain the term t and $\max_{\{t' \in d\}} N_{t'}$ is the maximum frequency of term t in the documents.

All TF and IDF variants in the above tables are tested for the TF and IDF WSs; for TF∗IDF weighting, all combinations of TF and IDF variants are tested. We have not employed any feature selection since features are already sparse and valuable, unlike in other text classification or clustering domains. To calculate the distance between record pairs, we have tested the distance metrics presented in Table 3.

As shown in Table 3, the range of each distance algorithm changes. Additionally, some of them do not have a finite

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

```
1.  Procedure ProductCodeMatching (featureList_R1, featureList_R2, Settings)
2.   begin
3.    initialize candidateCodeList CL1, CL2
4.    for each feature fi in featureList Ri
5.     if (length(fi) > Settings.minLength and hasChar(fi) and hasDigit(fi)) //First Stage
6.      then Add fi to CLi;
7.     endfor;
8.    boolean ProductCodeFound = false, number MatchCount = 0;
9.    for each feature fi in candidateCodeList CLi
10.    for each string si in Settings.specialCodeIgnoreFeaturesList //e.g. "hz", "inc"
11.     mi = remove si from fi;
12.     if (mi is a number)
13.      then remove fi from CLi;
14.    endfor;
15.   endfor;
16.   for each feature f1 in CL1
17.    for each feature f2 in CL2
18.     if (absolute (length(f1) - length(f2)) <= Settings.editDistanceProductCode) {
19.      minLength = minimum (length(f1), length(f2));
20.      minPairingLength = floor (minLength * Settings.minSubstringRatio);
21.      if (f1 startswith substring of (f2, minPairingLength)) {
22.       firstMod = substring of (f1, minPairingLength); //e.g. i54210u > 4210u
23.       secondMod = substring of (f2, minPairingLength); //e.g. i526 > 26
24.       if (firstMod[lastIndex] is a letter) //e.g. 4210u > 4210
25.        then firstMod = substring of (firstMod, length(firstMod) - 1);
26.       if (secondMod[lastIndex] is a letter)
27.        then secondMod = substring of (secondMod, length(secondMod) - 1);
28.       boolean ignoredBySerialNumber = false;
29.       if (firstMod is a number and secondMod is a number and firstMod != secondMod)
30.        then ignoredBySerialNumber = true; //e.g. slx301 vs slx241
31.       ProductCodeFound = true;
32.       editDistance = Levenshtein_Distance(f1, f2);
33.       if (Settings.editDistanceProductCode > editDistance and ignoredBySerialNumber == false)
34.        then MatchCount++;
35.       else if (f1 contains f2 or f2 contains f1)
36.         then MatchCount++;
37.      }
38.     }
39.     if (ProductCodeFound == false) { //Second Stage
40.      longestCommonString = LongestCommonSubstring(f1, f2);
41.      minLongestLength = minLength(f1, f2);
42.      if (Settings.minLCSRatioWordLength <=  minLongestLength) {
43.       minLongestLength = minLongestLength * Settings.minLongestCommonRatio;
44.       if (longestCommonString >= minLongestLength)
45.        then ProductCodeFound = true;
46.      }
47.     }
48.    endfor;
49.   endfor;
50.  if (MatchCount == 0 and ProductCodeFound == true)
51.   then return false;
52.  else return true;
```

FIGURE 3. Pseudocode of product code matching algorithm.

upper limit. Because of this reason, a separate range for each distance metric is needed to be determined during the experimentation.

Since each cluster can have multiple records, we have experimented with multiple linkage algorithms to calculate the final distance between cluster pairs. The linkage criteria used in this study are shown in Table 4, where A and B are the compared sets (clusters), and d is the chosen distance metric. For more information about linkage criteria, we refer to the survey published by Murtagh [82].

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

TABLE 1. Experimented TF variants.

Name of TF variant	Equation	Eq. #
Binary	$\begin{cases} 0 \text{ if term } t \text{ is not in document } d \\ 1 \text{ otherwise} \end{cases}$	(2)
Raw frequency	$f_{t,d} =$ frequency of term t in document d	(3)
Log normalization	$1 + \log\left(f_{t,d}\right)$	(4)
Double normalization	$0.5 + 0.5 \times \frac{f_{t,d}}{\max\left\{t' \in d\right\} f_{t',d}}$	(5)
BM25	$\frac{\left(k_1+1\right) \times f_{t,d}}{k_1 \times \left(\left(1-b\right)+b \times \frac{dl}{avg_dl}\right)}$	(6)

TABLE 2. Experimented IDF variants.

Name of IDF variant	Equation	Eq. #
Unary IDF	1	(7)
IDF	$\log \frac{N}{N_t}$	(8)
IDF smooth	$\log\left(1 + \frac{N}{N_t}\right)$	(9)
IDF max	$\log\left(1 + \frac{\max\left\{t' \in d\right\} N_{t'}}{N_t}\right)$	(10)
Probabilistic IDF	$\begin{cases} 0 \; if \; N = N_t \\ \log \frac{N-N_t}{N_t} \text{ otherwise} \end{cases}$	(11)

Another very controversial area of clustering is how the resulting clusters will be evaluated. There does not exist any widely accepted metric to measure the performance of the clustering process. Additionally, even the implementation of the same metric may differ in different studies. For example, for the F-measure metric, we have found at least five different algorithms that are used for its implementation. Since there is no single implementation of the same metric in the previous studies, the results of the previous works are not directly comparable if the used implementation is not clearly mentioned. According to our analysis, in many previous works, the implementation of the used evaluation metrics has not been presented.

When developing our system, we have used the F-measure algorithm presented by Manning *et al.* [88] to evaluate our results. However, all of the scores we present in this study are calculated by using '*Environment for Developing KDD-Applications Supported by Index-Structures (ELKI)*' open-source data mining software [89]. The settings of score calculation in ELKI are also essential. How we calculate the scores is presented in Fig. 4. Additionally, the command argument that is used to make performance evaluation in ELKI is shown as follows:

KDDCLIApplication -dbc.in 'input.arff' -algorithm clustering.meta.ExternalClustering -externalcluster.file 'inputClusteringVectorDumper.txt' -evaluator clustering.Evaluate

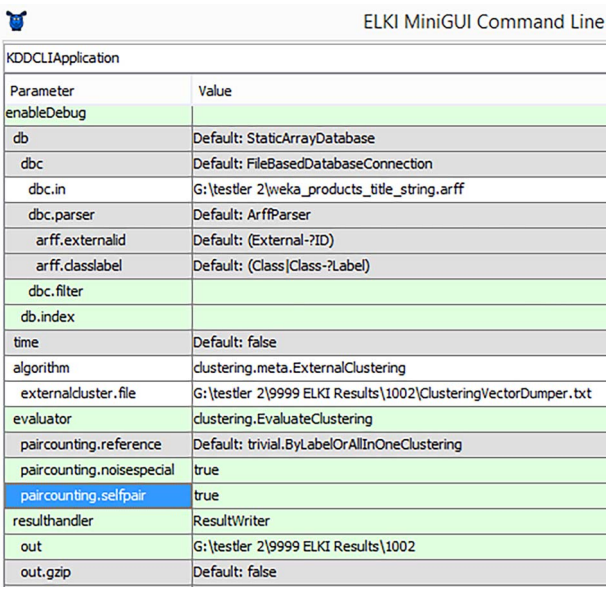


FIGURE 4. An example of the settings that are used to calculate clustering evaluation by ELKI.

Clustering -paircounting.noisespecial -paircounting.selfpair -resulthandler ResultWriter -out 'outputDirectory'

When the above settings are used, the F-measure under the *Pair counting measures* section of ELKI produces the same

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

TABLE 3. Experimented distance algorithms.

Distance metric	Formula	Range	Eq. #
Euclidean Distance, where q and p are feature weight vectors.	$d\left(q,p\right) = \sqrt{\left(\sum_{i=1}^{n}\left(q_i - p_i\right)^2\right)}$	$0-\infty$	(12)
Squared Euclidean Distance, where q and p are feature weight vectors.	$d\left(q,p\right) = \left(\sum_{i=1}^{n}\left(q_i - p_i\right)^2\right)$	$0-\infty$	(13)
Manhattan Distance (Taxicab Geometry), where q and p are feature weight vectors.	$d\left(q,p\right) = \left(\sum_{i=1}^{n}\left\|q_i - p_i\right\|\right)$	$0-\infty$	(14)
Maximum Distance (Chebyshev Distance), where q and p are feature weight vectors.	$d\left(q,p\right) = \max_i\left(\left\|q_i - p_i\right\|\right)$	$0-\infty$	(15)
Jaccard Distance, where A and B are string vectors. This algorithm is the same as *the Tanimoto Binary Distance*.	$d_J\left(A,B\right) = 1 - s_J\left(A,B\right) = 1 - \frac{\left\|A\cup B\right\| - \left\|A\cap B\right\|}{\left\|A\cup B\right\|}$	$0-1$	(16)
Pearson Correlation Distance, where x and y are data objects, and N is the total number of attributes [77].	$\text{Pearson}\left(x,y\right) = \frac{\sum xy - \frac{\sum x \sum y}{N}}{\sqrt{\left(\sum x^2 - \frac{\left(\sum x\right)^2}{N}\right)\left(\sum y^2 - \frac{\left(\sum y\right)^2}{N}\right)}}$	$0-2$	(17)
Average Kullback Leibler Divergence Distance, where a and b are feature weight vectors, and P is probability function [78].	$d\left(a,b\right) = \frac{\left(\sum_{i=1}^{n}\left(a_i\right) \times \log_2 P\left(a_i\right) + \sum_{i=1}^{n}\left(b_i\right) \times \log_2 P\left(b_i\right)\right)}{2\times\log_e 2}$	$0-\infty$	(18)
Cosine Distance, where q and p are feature weight vectors and all weights ≥ 0.	$d_C\left(q,p\right) = 1 - s_C\left(q,p\right) = 1 - \frac{\sum_{i=1}^{n}\left\|q_i p_i\right\|}{\sqrt{\sum_{i=1}^{n} {q_i}^2}\sqrt{\sum_{i=1}^{n} {p_i}^2}}$	$0-1$	(19)
Bray Curtis Distance, where q and p are feature weight vectors [79].	$d\left(q,p\right) = \frac{\sum_{i=1}^{n}\left\|q_i - p_i\right\|}{\sum_{i=1}^{n}\left\|q_i + p_i\right\|}$	$0-1$	(20)
Canberra Distance, where q and p are feature weight vectors.	$d\left(q,p\right) = \sum_i^n \frac{\left\|q_i - p_i\right\|}{\left\|q_i\right\| + \left\|p_i\right\|}$	$0-\infty$	(21)
Normalized Canberra Distance, where q and p are feature weight vectors.	$d\left(q,p\right) = \frac{1}{n}\sum_i^n \frac{\left\|q_i - p_i\right\|}{\left\|q_i\right\| + \left\|p_i\right\|}$	$0-1$	(22)
Hamming Distance, where i and j are objects, and k is the index of the respective variable reading y out of the total number of variables n [80].	$d^{HAD}\left(i,j\right) = \sum_{k=0}^{n-1}\left[y_{i,k} \neq y_{j,k}\right]$	$0-\infty$	(23)
Dice Distance, where A and B are strings.	$d_D\left(A,B\right) = 1 - s_D\left(A,B\right) = 1 - \frac{2\left\|A\cap B\right\|}{\left\|A\right\| + \left\|B\right\|}$	$0-1$	(24)
Tanimoto Non-Binary Distance, where A and B are feature weight vectors. This algorithm is the same as *Extended Jaccard Distance* [81].	$d_T\left(A,B\right) = 1 - s_T\left(A,B\right) = 1 - \frac{\sum_i\left(A_i B_i\right)}{\sum_i\left({A_i}^2\right) + \sum_i\left({B_i}^2\right) - \sum_i\left(A_i B_i\right)}$	$0-1$	(25)

result as the F-measure computed as follows:

$$F_\beta = \left(1+\beta^2\right) \times \frac{\text{precision} \times \text{recall}}{\left(\beta^2 \times \text{precision}\right) + \text{recall}}, \quad (31)$$

where $\beta = 1$ is used by Manning *et al.* [88]. We have found that *Pair counting measures* are the most appropriate approach for measuring the quality of the clustering in the record linkage area according to our empirical analysis.

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

TABLE 4. Experimented Linkage Criteria algorithms.

Name of the Linkage Criteria	Formula	Eq. #
Maximum or Complete-Linkage Clustering [83]	$\max\left\{d\left(a,b\right) : a \in A, b \in B\right\}$	(26)
Minimum or Single-Linkage Clustering [84]	$\min\left\{d\left(a,b\right) : a \in A, b \in B\right\}$	(27)
Mean or Average Linkage Clustering, or UPGMA [85]	$\frac{1}{\left\|A\right\|\left\|B\right\|}\sum_{a\in A}\sum_{b\in B} d\left(a,b\right)$	(28)
Centroid Linkage Clustering, or UPGMC [86], where c_S and c_T are the centroids of clusters S and T, respectively	$\left\|c_S - c_T\right\|$	(29)
Wards Linkage, where ESS is Error Sum of Squares, and X_{ijk} is the value for variable k in observation j belonging to cluster i [87].	$d\left(A,B\right) = ESS\left(A \cup B\right) - \left(ESS(A) + ESS(B)\right) ESS(A) = \sum_i \sum_j \sum_k \left\|X_{ijk} - \bar{x}_{ik}\right\|^2$	(30)

```
# Pair counting measures
Jaccard 0.2597482979162368
F1-Measure 0.41238126433016703
Precision 0.9862906384645516
Recall 0.26068951237188115
Rand 0.9951515479102199
ARI 0.4107734749769209
FowlkesMallows 0.5070657014611374
# Entropy based measures
NMI Joint 0.8102981093226196
NMI Sqrt 0.9000344584388756
# BCubed-based measures
F1-Measure 0.5836728501804221
Recall 0.9943126793795009
Precision 0.41307682074686025
# Set-Matching-based measures
F1-Measure 0.6774862144326119
Purity 0.5136548518303323
Inverse Purity 0.9947704822777517
# Editing-distance measures
F1-Measure 0.46126330065998566
Precision 0.38524113887274836
Recall 0.5746658919233004
# Gini measures
Mean +-0.3443 0.7036947500631808
```

FIGURE 5. An example clustering evaluation results generated by ELKI.

Therefore, all of the F-measures presented in this article are calculated by using ELKI's *Pair counting measures* approach. An example of evaluation results generated by ELKI is displayed in Fig. 5.

6. EXPERIMENTAL RESULTS AND DISCUSSIONS

All of the experiments are done on our personal home computer, which has the following specifications: Windows 8.1 (6.3) Enterprise Edition 64-bit operating system, 32 GB DDR3 RAM, Raid 0 SSD hard drive, Core i7-2600K CPU that is overlocked to 4.5GHZ. For programming our software, we have used the C# programming language by using .NET 4.5 framework.

Since there are thousands of different combinations of the above-listed term weighting, vectorization, distance measure, linkage criteria, etc., algorithms and hundreds of different parameter options, we have used an iterative empirical approach for conducting the tests. Additionally, some techniques such as spelling correction, blocking with an SVM-based classification, similarity-based thresholding that we have experimented, have failed to improve the performance. All of the experiments are conducted on datasets that have the price feature of the products.

The iterative empirical approach we propose is as follows: splitting all possible algorithm and parameter combinations into smaller logical stages, conducting experiments with these stages and then conducting the final experiments with the most promising configurations obtained in the previous stages. This approach still does not ensure the best possible outcome. However, it is one of the most effective approaches when it is impossible to experiment with all possible configurations and parameters.

In the first stage, we have experimented to find which combination of vector normalization, BOW transformation, the maximum price ratio of the price-matching system and the WS is promising. All combinations of the vector transform, TF or TF∗IDF WSs, TF variances, IDF variances and the BOW approaches presented in Section 4 are tested. However, these tests still require distance metric, distance threshold and the linkage criteria to be chosen. Also, our proposed matching system configurations are necessary. Therefore, it is not possible to find which combination performs best unless all combinations are tested. We have used the ones that we have found as the

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

TABLE 5. Most promising configuration of the first stage experiments.

Method name	Methodology
Linkage criteria	Average Linkage Clustering (UPGMA)
Distance metric	Squared Euclidean Distance
Vector normalization	L2
WS	TF∗IDF
IDF variant	IDF Smooth
TF variant	BM25_TF
Max distance to be paired threshold	1.45
Maximum price ratio	1.5
BOW method	Word-based 1-gram + character-based 2-gram

TABLE 6. Experimental results of distance metrics, linkage criteria and distance thresholds for record linkage task

Configuration *Dataset: Price feature having dataset*			F	ARI	P	R
Distance metric	Linkage criteria	DT				
Average Kullback Leibler Divergence Distance	Complete	1.88	38.98	38.46	29.99	55.64
Bray Curtis Distance	Average	0.69	95.97	95.94	98.17	93.87
Canberra Distance	Average	54.0	79.19	79.05	77.45	81.00
Cosine Distance	Centroid	0.61	**96.45**	**96.42**	98.10	94.85
Dice Distance	Average	0.60	95.05	95.02	96.86	93.31
Euclidean Distance	Average	1.20	96.09	96.06	96.62	95.55
Tanimoto Non-Binary Distance	Centroid	0.77	96.22	96.20	97.70	94.79
Hamming Distance	Wards	2.0	54.57	54.16	40.50	83.60
Jaccard Distance	Average	0.75	94.88	94.84	96.77	93.06
Manhattan Distance	Wards	0.80	92.68	92.63	96.90	88.80
Maximum Distance	Wards	0.79	92.84	92.79	97.31	88.76
Pearson Correlation Distance	Centroid	1.04	96.13	96.10	**99.07**	93.35
Squared Euclidean Distance	Average	1.44	96.23	96.21	96.78	**95.69**
Normalized Canberra Distance	Average	0.77	93.88	93.84	97.19	90.78

most useful during the initial system development for these untested configurations and parameters. After our experiments were done in the first stage, we have chosen the most promising configuration to experiment with the other stages due to lack of time and resources. The most promising configuration we have found during the first stage of experimental evaluation is shown in Table 5, which uses squared Euclidean distance and average linkage clustering with TF∗IDF WS for word 1-gram and character 2-gram features. The results displayed in Tables 6–8 are the best performing results of the tests run on the dataset, which has the price attribute for all the products.

In the second stage of experimenting, we have tested every configuration of distance algorithms and the linkage criteria shown in Tables 3 and 4, respectively. Since the range of each distance algorithm changes, the algorithms work only with the proper distance parameter. Therefore, we have conducted the experiments with a different range of distance thresholds for each distance metric. We have checked the records having the longest distance in our dataset, and based on that distance value, we have set the upper limit for the ones that have infinite range. Then, we have split this range between 0 and the determined upper limit to 100 equal intervals, and then, each interval is tested. For the distance metrics with an upper limit, such as Dice Distance, which is always between 0 and 1, we have applied the same approach. For example, for the Euclidean Distance metric, the farthest distance value between records in our dataset is 1.4. Thus, we have decided the upper limit as 2.0 and tested all values between 0 and 2 with 100 equal intervals. This approach ensured that each algorithm was tested as much as equally and provided more objective results. The top-performing configuration results of the second stage experiments for each distance metric are shown in Table 6, where DT is the distance threshold, F is F-measure, P is precision, R is Recall and ARI is Adjusted

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

TABLE 7. Results of parameter experiments for the product code detection system.

Parameter	Tested values	Best performing value
minLength	3, 4, 5, 6, 7	4
editDistanceProductCode	0, 1, 2, 3, 4, 5	2
minSubstringRatio	20%, 30%, 40%, 50%, 60%	20%
minLCSRatioWordLength	3, 4, 5, 6, 7, 8	8
minLongestCommonRatio	40%, 50%, 60%, 70%, 80%	40%

TABLE 8. Results of the final parameter configuration experiments.

Configuration	F	ARI	P	R
DM: Jaccard Distance LC: Average Linkage Clustering, DT: 0.75 ML: 4, DPC: 2, SR: 10%, LRW: 8, LCR: 30%	95.67	95.64	97.98	93.47
DM: Pearson Correlation Distance LC: Centroid Linkage Clustering, DT: 1.04 ML: 4, DPC: 2, SR: 30%, LRW: 8, LCR: 30%	96.48	96.46	**99.64**	93.51
DM: Cosine Distance—**Best** LC: Centroid Linkage Clustering, DT: 0.61 ML: 4, DPC: 2, SR: 20%, LRW: 8, LCR: 30%	**97.12**	**97.10**	99.57	**94.79**
DM: Bray Curtis Distance LC: Average Linkage Clustering, DT: 0.69 ML: 4, DPC: 2, SR: 10%, LRW: 8, LCR: 30%	96.52	96.50	98.87	94.28
DM: Normalized Canberra Distance LC: Average Linkage Clustering, DT: 0.77 ML: 4, DPC: 2, SR: 10%, LRW: 8, LCR: 30%	94.88	94.84	98.60	91.42
DM: Dice Distance LC: Average Linkage Clustering, DT: 0.60 ML: 4, DPC: 2, SR: 10%, LRW: 8, LCR: 30%	95.67	95.64	97.98	93.47
DM: Tanimoto Non-Binary Distance LC: Centroid Linkage Clustering, DT: 0.77 ML: 4, DPC: 2, SR: 10%, LRW: 8, LCR: 30%	96.92	96.90	99.19	94.76

Rand Index. The global best results are written in boldface in Table 6.

In the third stage, we have experimented with the *minLength*, *editDistanceProductCode*, *minSubstringRatio*, *minLCSRatioWordLength* and *minLongestCommonRatio* parameters of the product code detection algorithm that is shown in Fig. 3. This time, we have used the Cosine Distance configuration that we have presented in Table 6. The best results we have obtained in this new third stage experiments are 97.12% F-measure, 97.10% ARI, 99.57% Precision and 94.79% Recall. The details of the third stage product code experiments are shown in Table 7.

Then, we have experimented with the most promising configurations that we have obtained in the first three stages of the experiments. We have selected the Jaccard Distance, Pearson Correlation Distance, Cosine Distance, Bray Curtis Distance, Normalized Canberra Distance, Dice Distance and Tanimoto Non-Binary Distance configurations since they all perform with over 85% precision, and their ranges are finite. Thus, they are easier to apply to different domains or datasets. As a result of the final experiments, we have obtained the best results shown in Table 8. In Table 8, the WS is TF∗IDF, where the TF variant is BM25, and the IDF variant is IDF Smooth; vector normalization is L2. In Table 8, the abbreviations ML, DPC, SR, LRW and LCR mean *minLength*, *editDistanceProductCode*, *minSubstringRatio*, *minLCSRatioWordLength* and *minLongestCommonRatio*, respectively. According to the results presented in Table 8, we decided to use the configuration given in the third row of the table.

As the next step, we have experimented on our macro-average dataset with the best-performing configuration. Additionally, we have tested many state-of-the-art clustering algorithms by using the latest ELKI 0.7.1 framework on the macro-average dataset. The results we have obtained on the macro-average dataset are presented in Table 9. In Table 9, *Minclusters* and *K* are the numbers of clusters in the clustered

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

TABLE 9. Experimental results of clustering algorithms over the macro-average dataset.

Configuration	F-measure		ARI		Precision		Recall		RT
	Raw	TIF	Raw	TIF	Raw	TIF	Raw	TIF	
Our System: DM: Cosine Distance	**91.68**	**95.07**	**91.62**	**95.03**	**91.06**	**97.03**	**90.09**	**91.27**	49.6
LC: Centroid Linkage Clustering, DT:	**93.87**	**96.25**	**93.83**	**96.23**	**95.29**	**99.23**	**92.51**	**93.47**	
0.61, ML: 4, DPC: 2, SR: 20%, LRW:	**95.93**	**97.54**	**95.9**	**97.52**	**97.6**	**99.87**	**95.01**	95.84	
8, LCR: 30%									
Affinity Propagation [90]	32.06	76.68	31.39	76.53	20.53	81.84	63.02	68.83	27.7
	41.38	79.95	40.84	79.82	29.83	86	68.62	74.75	
	52.39	84.52	52	84.42	41.72	89.53	78.3	82.56	
Extract Flat Clustering From Hierarchy	2.95	61.6	1.7	61.25	1.51	45.78	70.24	90.35	22.5
+ AGNES [91] + Single Linkage +	4.97	72.48	3.75	72.26	2.57	59.55	74.07	93.13	
Squared Euclidean Distance,	7.36	81.42	6.16	81.27	3.87	71.76	80.47	94.93	
Minclusters									
Extract Flat Clustering From Hierarchy	35.56	84.59	34.93	84.49	23.9	82.76	62.06	84.34	22.3
+ AGNES + Complete Linkage +	53.8	87.87	53.43	87.79	44.03	87.58	70.51	88.19	
Squared Euclidean Distance,	67.48	91.95	67.25	91.9	63.07	92.31	76.31	93.79	
Minclusters									
Extract Flat Clustering From Hierarchy	14.03	86.51	12.98	86.41	7.7	82.22	74.11	90.04	22.2
+ AGNES + Average Linkage +	21.7	89.12	20.81	89.04	12.65	85.47	78.23	93.12	
Squared Euclidean Distance,	31.87	92.08	31.14	92.03	19.89	89.78	83.21	**96.2**	
Minclusters									
Extract Flat Clustering From Hierarchy	1.96	2.31	0.74	1.08	1	1.17	65.82	76.62	22.3
+ AGNES + Centroid + Squared	2.32	6.62	1.05	5.42	1.18	5.3	69.34	80.96	
Euclidean Distance, Minclusters	2.67	84.48	1.35	84.38	1.36	85.22	73.67	86.2	
Extract Flat Clustering From Hierarchy	63.55	72.39	63.32	72.23	65.54	89.9	58.73	60.21	22.1
+ AGNES + Wards Linkage +	68.05	77.12		76.99	71.44	92.46	65.05	66.17	
Squared Euclidean Distance,	74.34	82.36	67.85	82.27	79.01	94.84	70.99	73.49	
Minclusters			74.19						
K-Means Lloyd [92], K	12.34	54.71	11.37	54.44	7.04	59.07	45.22	49.01	37.4
	22.95	61.08	22.15	60.84	14.95	66.91	53.17	56.25	
	39.5	69.22	38.97	69.04	30.12	75.52	61.14	64.26	
K-Means Sort [93], K	12.85	54.96	11.85	54.7	7.25	55.79	46.32	48.87	24
	20.23	61.08	19.38	60.85	12.63	67.17	53.55	56.07	
	35.15	68.23	34.56	68.04	25.62	75.86	63.21	64.88	
K-Means Compare [93], K	15.42	55.19	14.46	54.93	9.01	60.54	46.61	49.24	21.3
	23.08	61.84	22.28	61.61	14.98	67.6	52.78	57.05	
	39.48	67.93	38.94	67.74	29.49	74.8	62.18	64.26	
K-Means Hamerly [94], K	28.73	57.8	28.02	57.55	19.27	58.61	47.13	51.47	29
	42.55	62.59	42.08	62.36	35.36	67.74	54.66	58.23	
	53.86	69.55	53.54	69.39	51	74.7	61.48	66.75	
K-Means Elkan [95], K	26.84	55.77	26.12	55.5	18.13	59.64	48.93	50.93	**7.8**
	41.47	62.67	40.98	62.44	33.96	67.9	54.91	58.24	
	56.13	70.9	55.83	70.72	53.55	75.06	62.35	67.18	
K-Medians Lloyd [96], K	9.79	34.88	8.75	34.32	5.49	25.81	43.49	50.51	52.3
	16.37	50.09	15.44	49.72	9.9	45.27	50.48	57.15	
	26.79	64.54	26.07	64.31	17.92	65.81	59.92	65.93	
K-Means MacQueen [97], K	8.97	7.3	7.87	6.25	4.91	3.89	50	54.41	23.2
	14.19	11.75	13.2	10.7	8.15	6.53	56.91	60.23	
	23.67	17.11	22.84	16.13	14.61	9.83	64.75	66.19	

Continued

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

TABLE 9. Continued.

Configuration	F-measure		ARI		Precision		Recall		RT
	Raw	TIF	Raw	TIF	Raw	TIF	Raw	TIF	
K-Medoids PAM [98], K	33.16	67.01	32.56	66.83	24.79	85.49	50.08	54.84	24.9
	43.87	71.98	43.42	71.83	36.5	88.23	55.56	60.82	
	55.29	76.82	54.98	76.7	53.35	90.99	61.99	68.46	
K-Medoids EM [89], K	32.18	81.77	31.5	81.65	20.9	80.92	65.15	78.86	58.9
	48.26	85.87	47.81	85.78	37.07	85.93	71.49	85.88	
	63.96	89.4	63.69	89.33	57.36	91.7	77.54	91.14	
K-Means Batched Lloyd, K	9.01	11.49	7.84	10.37	4.96	6.4	47.48	52.46	26.1
	14.38	19.23	13.4	18.33	8.29	11.47	56	61.33	
	23.39	28.72	22.56	27.99	14.33	18.86	63.58	69.05	

dataset; *TIF* is TF∗IDF WS, which is shown in Table 5; *Raw* is the raw term frequency where vector normalization is none and BOW methodology is word-based 1-gram. The dataset files generated based on the Raw and TIF configurations are available in the Github repository. Furthermore, in Table 9, in each result cell, the first value is the minimum, the second value is the average and the third value is the maximum value of the obtained results in the tests conducted with the respected configuration. For example, for the K-Means Sort algorithm, 12.85 is the minimum, 20.23 is the average and 35.15 is the maximum F-measure where raw term frequency is used. We only present non-default parameters of the algorithms in the configuration except for Agglomerative Nesting (AGNES), also known as the HAC. In AGNES, we also presented the linkage criteria and the distance metric. The min, max and average results of clustering experiments conducted on the macro-average dataset are presented in Table 9, where RT is the average algorithm run-time for each test in seconds.

When the results in Table 9 are interpreted, our proposed framework outperforms all of the previously proposed state-of-the-art systems. However, our algorithm is also the third slowest one even though we apply multi-tasking for the distance calculation process between the candidates and the new record. After analyzing the run times of all tasks by using the Microsoft Visual Studio profiler, we have noticed that the most time-consuming task is the one that assigns numerical weights to the tokens in our proposed system. Since our system works in incremental mode, we have to re-assign weights to the compared records set in each clustering iteration. However, other algorithms work in batch mode, and they are provided with the token set that numerical weights are already assigned. Once we ignore the time required for calculating the weights, the running time of our algorithm reduces to 10.2 from 49.6 seconds, and it becomes the second-fastest. We plan to improve the performance of the weight computation task as a future study.

In Table 9, we see that AGNES used with Average linkage performs the best among the other algorithms. It achieves an 89.12% average F-measure, which is the closest to our results. However, we have to note that the other algorithms have several very important leverages. They are provided with the entire dataset, unlike our method. Our system is incremental; thus, it clusters each record one by one. Additionally, they are provided with the exact number of clusters. Further analysis reveals that the TF∗IDF weighting system performs much better than the Raw term frequency WS. One of the primary reasons for this is that since the entire dataset is provided, the TF∗IDF WS can tokenize the records with better weights. However, unlike the previously proposed systems, our system can obtain significant performance when the raw term frequency is used, because our proposed matching system significantly prevents incorrect-merges.

7. CONCLUSION

In this study, we have presented a novel record linkage system for E-commerce products. Our system outperforms similar systems that are previously proposed. To the best of our knowledge, our study is unique when all of its properties are taken into consideration. The product code matching system and the other matching methods that we have proposed significantly improve the success rate of the record linkage task. Moreover, we have systematically experimented with many different state-of-the-art metrics, and we have shown which metrics are more promising in the record linkage task of E-commerce products. Furthermore, as far as our knowledge, this study is the first one to publish a public E-commerce product dataset[6] in this area.

Although our system outperforms similar systems in the current literature, there is still space for improvement. In our experimental evaluation, we have compared a set of promising combinations of vectorization, WS, linkage criteria and

6 https://github.com/FurkanGozukara/Record-Linkage

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

distance metric methods, with a set of threshold values. As there exist a huge number of distinct combinations of the tested methods and thresholds, as future work, experimental evaluation can be extended by trying the missing combinations to further improve the performance of the proposed system.

Additionally, the automatic generation of brand word lists and conflicting words can be investigated as a future work. Recently proposed Euclidean distance-based clustering techniques, such as critical distance clustering, can be applied to our problem. Moreover, there are still unexperimented text representation techniques such as word embeddings by using Word2vec, FastText, GloVe, BERT, etc. Finally, deep clustering techniques can be applied as another future work.

AVAILABILITY OF DATA

The data underlying this article are available in 'Turkish E-commerce Products Dataset For Record Linkage' at https://github.com/FurkanGozukara/Record-Linkage and can be accessed with its URL.

FUNDING

Scientific and Technological Research Council of Turkey (TÜBİTAK) scholarship 2211-C.

REFERENCES

[1] U. C. Bureau. *Quarterly Retail E-Commerce Sales*. November 19, 2020. [Online]. https://www.census.gov/retail/mrts/www/data/pdf/ec_current.pdf (Accessed December 6, 2020).

[2] Pathak, B. (2010) A Survey of the Comparison Shopping Agent-Based Decision Support Systems. *J. Electron. Commer. Res.*, 11, 178.

[3] Kannan, A., Givoni, I.E., Agrawal, R. and Fuxman, A. Matching Unstructured Product Offers to Structured Product Specifications. In *Proc. of the 17th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 404–412, 2011.

[4] Elfeky, M.G., Verykios, V.S. and Elmagarmid, A.K. TAILOR: A Record Linkage Toolbox. In *Proc. of 18th Int. Conf. on Data Engineering*, pp. 17–28, 2002.

[5] Jin, L., Li, C. and Mehrotra, S. Efficient Record Linkage in Large Data Sets. In *Proc. of Eighth Int. Conf. on Database Systems for Advanced Applications, 2003.(DASFAA 2003)*, pp. 137–146, 2003.

[6] Bilenko, M., Kamath, B. and Mooney, R.J. Adaptive Blocking: Learning to Scale Up Record Linkage. In *Sixth Int. Conf. on Data Mining (ICDM'06)*, pp. 87–96, 2006.

[7] Fellegi, I.P. and Sunter, A.B. (1969) A Theory for Record Linkage. *J. Am. Stat. Assoc.*, 64, 1183–1210.

[8] Christen, P. (2012) A Survey of Indexing Techniques for Scalable Record Linkage and Deduplication. *IEEE Trans. Knowl. Data Eng.*, 24, 1537–1555.

[9] Sarawagi, S. and Bhamidipaty, A. Interactive Deduplication using Active Learning. In *Proc. of the Eighth ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 269–278, 2002.

[10] Cohen, W., Ravikumar, P. and Fienberg, S. (2003) A Comparison of String Metrics for Matching Names and Records. *Kdd Workshop on Data Cleaning and Object Consolidation*, 3, 73–78.

[11] Cohen, W.W. (1998) Integration of Heterogeneous Databases without Common Domains using Queries based on Textual Similarity. *ACM SIGMOD Record*, 27, 201–212.

[12] Florescuand, D. An Extensible Framework for Data Cleaning. In *Proc. of the 16th Int. Conf. on Data Engineering*, p. 312, 2000.

[13] Rahm, E. and Do, H.H. (2000) Data Cleaning: Problems and Current Approaches. *IEEE Data Eng. Bull.*, 23, 3–13.

[14] Chaudhuri, S., Ganjam, K., Ganti, V. and Motwani, R. Robust and Efficient Fuzzy Match for Online Data Cleaning. In *Proc. of the 2003 ACM SIGMOD Int. Conf. on Management of Data*, pp. 313–324, 2003.

[15] Bilenko, M. and Mooney, R.J. On Evaluation and Training-Set Construction for Duplicate Detection. In *Proc. of the KDD-2003 Workshop on Data Cleaning, Record Linkage, and Object Consolidation*, pp. 7–12, 2003.

[16] Elmagarmid, A.K., Ipeirotis, P.G. and Verykios, V.S. (2007) Duplicate Record Detection: A Survey. *IEEE Trans. Knowl. Data Eng.*, 19, 1–16.

[17] Monge, A. and Elkan, C. (1997) An Efficient Domain-Independent Algorithm for Detecting Approximately Duplicate Database Records. *Proc. of the SIGMOD*.

[18] Bilenko, M. and Mooney, R.J. Adaptive Duplicate Detection using Learnable String Similarity Measures. In *Proc. of the Ninth ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 39–48, 2003.

[19] Aizawa, A. and Oyama, K. A Fast Linkage Detection Scheme for Multi-Source Information Integration. In *Int. Workshop on Challenges in Web Information Retrieval and Integration*, pp. 30–39, 2005.

[20] Bhattacharya, I. and Getoor, L. (2007) Collective Entity Resolution in Relational Data. *ACM Trans. Knowl. Discov. Data (TKDD)*, 1, 5.

[21] Christen, P., Gayler, R. and Hawking, D. Similarity-Aware Indexing for Real-Time Entity Resolution. In *Proc. of the 18th ACM Conf. on Information and Knowledge Management*, pp. 1565–1568, 2009.

[22] Whang, S.E., Menestrina, D., Koutrika, G., Theobald, M. and Garcia-Molina, H. Entity Resolution with Iterative Blocking. In *Proc. of the 2009 ACM SIGMOD Int. Conf. on Management of Data*, pp. 219–232, 2009.

[23] Bhattacharya, I. and Getoor, L. (2006) A Latent Dirichlet Model for Unsupervised Entity Resolution. *SDM*, 5, 59.

[24] Dong, X., Halevy, A. and Madhavan, J. Reference Reconciliation in Complex Information Spaces. In *Proc. of the 2005 ACM SIGMOD Int. Conf. on Management of Data*, pp. 85–96, 2005.

[25] Hernández, M.A. and Stolfo, S.J. (1995) The Merge/Purge Problem for Large Databases. *ACM Sigmod Record*, 24, 127–138.

[26] Tejada, S., Knoblock, C.A. and Minton, S. Learning Domain-Independent String Transformation Weights for High Accuracy Object Identification. In *Proc. of the Eighth ACM SIGKDD Int.*

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

Conf. on Knowledge Discovery and Data Mining, pp. 350–359, 2002.

[27] Singla, P. and Domingos, P. Object Identification with Attribute-Mediated Dependences. In *European Conf. on Principles of Data Mining and Knowledge Discovery*, pp. 297–308, 2005.

[28] Shen, W., Li, X. and Doan, A. (2005) Constraint-Based Entity Matching. *AAAI*, 862–867.

[29] Pasula, H., Marthi, B., Milch, B., Russell, S. and Shpitser, I. (2002) Identity Uncertainty and Citation Matching. In *Advances in Neural Inf. Processing Systems*, pp. 1401–1408.

[30] McCallum, A., Nigam, K. and Ungar, L.H. Efficient Clustering of High-Dimensional Data Sets with Application to Reference Matching. In *Proc. of the Sixth ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 169–178, 2000.

[31] Cohen, W.W., Kautz, H. and McAllester, D. Hardening Soft Information Sources. In *Proc. of the Sixth ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 255–259, 2000.

[32] Li, X., Morie, P. and Roth, D. (2004) Robust Reading: Identification and Tracing of Ambiguous Names. *DTIC Document*.

[33] Yan, S., Lee, D., Kan, M.-Y. and Giles, L.C. Adaptive Sorted Neighborhood Methods for Efficient Record Linkage. In *Proc. of the 7th ACM/IEEE-CS Joint Conf. on Digital Libraries*, pp. 185–194, 2007.

[34] Köpcke, H. and Rahm, E. (2010) Frameworks for Entity Matching: A Comparison. *Data Knowl. Eng.*, 69, 197–210.

[35] Koudas, N., Sarawagi, S. and Srivastava, D. Record Linkage: Similarity Measures and Algorithms. In *Proc. of the 2006 ACM SIGMOD Int. Conf. on Management of Data*, pp. 802–803, 2006.

[36] Winkler, W.E. (2006) Overview of Record Linkage and Current Research Directions. *Bureau of the Census*.

[37] Hall, R. and Fienberg, S.E. Privacy-Preserving Record Linkage. In *Int. Conf. on Privacy in Statistical Databases*, pp. 269–283, 2010.

[38] Aggarwal, C.C. and Zhai, C. (2012) A Survey of Text Clustering Algorithms. In *Mining Text Data*, pp. 77–128.

[39] Köpcke, H., Thor, A. and Rahm, E. (2010) Evaluation of Entity Resolution Approaches on Real-World Match Problems. *Proc. of the VLDB Endowment*, 3, 484–493.

[40] Christen, P. (2008) Febrl: A Freely Available Record Linkage System with a Graphical User Interface. In *Proc. of the Second Australasian Workshop on Health Data and Knowledge Management-Volume 80*, pp. 17–25.

[41] Cohen, W.W. and Richman, J. Learning to Match And Cluster Large High-Dimensional Data Sets for Data Integration. In *Proc. of the Eighth ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 475–480, 2002.

[42] Grannis, S.J., Overhage, J.M., Hui, S. and McDonald, C.J. (2003) Analysis of a Probabilistic Record Linkage Technique without Human Review. In *AMIA Annual Symposium Proceedings*, p. 259.

[43] Hotho, A., Staab, S. and Stumme, G. Ontologies Improve Text Document Clustering. In *Third IEEE Int. Conf. on Data Mining, 2003*, pp. 541–544, 2003.

[44] Fung, B.C., Wang, K. and Ester, M. (2003) Hierarchical Document Clustering Using Frequent Itemsets. *SDM*, 3, 59–70.

[45] Faloutsos, C. and Lin, K.-I. FastMap: A Fast Algorithm for Indexing, Data-Mining and Visualization of Traditional and Multimedia Datasets. In *SIGMOD '95 Proc. of the 1995 ACM SIGMOD Int. Conf. on Management of Data*, pp. 163–174, 1995.

[46] Hjaltason, G.R. and Samet, H. (1998) Incremental Distance Join Algorithms for Spatial Databases. *ACM SIGMOD Record*, 27, 237–248.

[47] Bhattacharya, I. and Getoor, L. Iterative Record Linkage for Cleaning and Integration. In *Proc. of the 9th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery*, pp. 11–18, 2004.

[48] Cui, X., Potok, T.E. and Palathingal, P. Document Clustering using Particle Swarm Optimization. In *Proc. 2005 IEEE Swarm Intelligence Symposium, 2005. SIS 2005*, pp. 185–191, 2005.

[49] Bilenko, M., Basil, S. and Sahami, M. Adaptive Product Normalization: Using Online Learning for Record Linkage in Comparison Shopping. In *Fifth IEEE Int. Conf. on Data Mining (ICDM'05)*, 2005.

[50] Shen, W., DeRose, P., Vu, L., Doan, A. and Ramakrishnan, R. Source-Aware Entity Matching: A Compositional Approach. In *IEEE 23rd Int. Conf. on Data Engineering*, pp. 196–205, 2007.

[51] Inan, A., Kantarcioglu, M., Bertino, E. and Scannapieco, M. A Hybrid Approach to Private Record Linkage. In *IEEE 24th Int. Conf. on Data Engineering*, pp. 496–505, 2008.

[52] Hastie, T., Tibshirani, R. and Friedman, J. (2009) The Elements of Statistical Learning. In *Springer Series in Statistics*, p. 506.

[53] Kim, H.-s. and Lee, D. HARRA: Fast Iterative Hashed Record Linkage for Large-Scale Data Collections. In *Proc. of the 13th Int. Conf. on Extending Database Technology*, pp. 525–536, 2010.

[54] Rastogi, V., Dalvi, N. and Garofalakis, M. (2011) Large-Scale Collective Entity Matching. *Proc. VLDB Endow.*, 4, 208–218.

[55] Kannan, A., Talukdar, P.P., Rasiwasia, N. and Ke, Q. Improving Product Classification using Images. In *IEEE 11th Int. Conf. on Data Mining*, pp. 310–319, 2011.

[56] Böhm, C., de Melo, G., Naumann, F. and Weikum, G. LINDA: Distributed Web-of-Data-Scale Entity Matching. In *Proc. of the 21st ACM Int. Conf. on Information and Knowledge Management*, pp. 2104–2108, 2012.

[57] Gokhale, C., Das, S., Doan, A., Naughton, J.F., Rampalli, N., Shavlik, J. and Zhu, X. Corleone: Hands-Off Crowdsourcing for Entity Matching. In *Proc. of the 2014 ACM SIGMOD Int. Conf. on Management of Data*, pp. 601–612, 2014.

[58] Zhang, D., Rubinstein, B.I. and Gemmell, J. (2015) Principled Graph Matching Algorithms for Integrating Multiple Data Sources. *IEEE Trans. Knowl. Data Eng.*, 27, 2784–2796.

[59] Collins, M. Discriminative Training Methods for Hidden Markov Models: Theory and Experiments with Perceptron Algorithms. In *Proc. of the ACL-02 Conf. on Empirical Methods in Natural Language Processing-Volume 10*, pp. 1–8, 2002.

[60] Sahoo, N., Callan, J., Krishnan, R., Duncan, G. and Padman, R. Incremental Hierarchical Clustering of Text Documents. In *Proc. of the 15th ACM Int. Conf. on Information and Knowledge Management*, pp. 357–366, 2006.

[61] Köpcke, H., Thor, A., Thomas, S. and Rahm, E. Tailoring Entity Resolution for Matching Product Offers. In *Proc. of the 15th Int. Conf. on Extending Database Technology*, pp. 545–550, 2012.

[62] Petrovski, P., Bryl, V. and Bizer, C. Integrating Product Data from Websites Offering Microdata Markup. In *Proc. of the 23rd Int. Conf. on World Wide Web*, pp. 1299–1304, 2014.

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021

[63] Isele, R. and Bizer, C. Learning Linkage Rules using Genetic Programming. In *Proc. of the 6th Int. Conf. on Ontology Matching-Volume 814*, pp. 13–24, 2011.
[64] Primpeli, A. and Bizer, C. Robust Active Learning of Expressive Linkage Rules. In *WIMS2019: Proc. of the 9th Int. Conf. on Web Intelligence, Mining and Semantics*, Seoul, Republic of Korea, 2019.
[65] Ristoski, P., Petrovski, P., Mika, P. and Paulheim, H. (2018) A Machine Learning Approach for Product Matching and Categorization. *Semantic Web*, 9, 707–728.
[66] Quoc, L. and Tomas, M. Distributed Representations of Sentences and Documents. In *Proc. of the 31st Int. Conf. on Machine Learning*, 2014.
[67] Luo, Y., Ma, J. and Chi, L. (2020) Entity Name Recognition of Cross-Border e-Commerce. *Electron. Commer. Res.*, 20, 405–426.
[68] Li, J., Dou, Z., Zhu, Y., Zuo, X. and Wen, J.-R. (2020) Deep Cross-Platform Product Matching in e-Commerce. *Inf. Retr. J.*, 23, 136–158.
[69] Gözükara, F. and Özel, S.A. (2017) Focused Web Crawler Development Challenges: Eccrawler. *Int. J. Comput. Sci. Eng.*, 6, 1–34.
[70] Rokach, L. and Maimon, O. (2005) Clustering Methods. In *Data Mining and Knowledge Discovery Handbook*, pp. 321–352.
[71] Berkhin, P. (2006) A Survey of Clustering Data Mining Techniques. In *Grouping multidimensional data*, pp. 25–71. Springer, Berlin Heidelberg.
[72] Sriram, B. (2010) Short text classification in Twitter to improve information filtering. In *MS Thesis, Computer Science and Engineering, The Ohio State University*.
[73] Cavnar, W.B. and Trenkle, J.M. (1994) *N-Gram-Based Text Categorization*, pp. 161–175. Environmental Research Institute of Michigan, Ann Arbor, MI 48113–4001.
[74] Salton, G., Wong, A. and Yang, C.-S. (1975) A Vector Space Model for Automatic Indexing. *Commun. ACM.*, 18, 613–620.
[75] Singhal, A. (2001) Modern Information Retrieval: A Brief Overview. *IEEE Data Eng. Bull.*, 24, 35–43.
[76] Paltoglou, G. and Thelwall, M. (2010) A Study of Information Retrieval Weighting Schemes for Sentiment Analysis. In *Proc. of the 48th Annual Meeting of the Association for Computational Linguistics*, pp. 1386–1395.
[77] Philip, S. *Discussion of Similarity Metrics, Pearson Correlation Coefficient*. 2010. [Online]. http://mines.humanoriented.com/classes/2010/fall/csci568/portfolio_exports/sphilip/pear.html (accessed June 22, 2016).
[78] Sutton, C. Returns the KL divergence. *Univ. of Massachusetts Amherst, Computer Science Dept.*, December 22, 2007. [Online]. http://www.java2s.com/Code/Java/Development-Class/ReturnstheKLdivergenceKp1p2.htm (accessed June 22, 2016).
[79] Souza, C. *BrayCurtis.cs*. Accord.NET on GitHub, June 1, 2016. [Online]. https://github.com/accord-net/framework/blob/development/Sources/Accord.Math/Distances/BrayCurtis.cs (accessed June 22, 2016).
[80] Schulz, J. *Hamming distance*. www.code10.info, May 15, 2008. [Online]. http://www.code10.info/index.php?option=com_content&view=article&id=59:hamming-distance&catid=38:cat_coding_algorithms_data-similarity&Itemid=57 (accessed June 22, 2016).
[81] Karlsson, C. *Similarity and Dissimilarity Measures*. http://inside.mines.edu, August 28, 2012. [Online]. http://web.simmons.edu/~benoit/lis642/Similarity%20and%20Dissimilarity%20Measures.pdf (accessed June 22, 2016).
[82] Murtagh, F. (1983) A Survey of Recent Advances in Hierarchical Clustering Algorithms. *Comput. J.*, 26, 354–359.
[83] Defays, D. (1977) An Efficient Algorithm for a Complete Link Method. *Comput. J.*, 20, 364–366.
[84] Sibson, R. (1973) SLINK: An Optimally Efficient Algorithm for the Single-Link Cluster Method. *Comput. J.*, 16, 30–34.
[85] Sokal, R. and Michener, C. (1958) A Statistical Method for Evaluating Systematic Relationships. *Univ. Kans. Sci. Bull.*, 38, 1409–1438.
[86] Jain, A.K. and Dubes, R.C. (1988) *Algorithms for Clustering Data*. Prentice-Hall, Inc.
[87] Wiesner, A. *Ward's Method*. Penn State Science, 2016. [Online]. http://sites.stat.psu.edu/∼ajw13/stat505/fa06/19_cluster/09_cluster_wards.html (accessed June 24, 2016).
[88] Manning, C.D., Raghavan, P. and Schütze, H. (2009) *An Introduction to Information Retrieval Online edition*, pp. 1–544. Cambridge University Press Cambridge, England.
[89] Schubert, E., Koos, A., Emrich, T., Züfle, A., Schmid, K.A. and Zimek, A. (2015) A Framework for Clustering Uncertain Data. *Proc. VLDB Endow.*, 8, 1976–1979.
[90] Frey, B.J. and Dueck, D. (2007) Clustering by Passing Messages between Data Points. *Science*, 315, 972–976.
[91] Kaufman, L. and Rousseeuw, P.J. (2008) Agglomerative Nesting (Program AGNES). *Finding Groups in Data: An Introduction to Cluster Analysis*, 199–252.
[92] Lloyd, S. (1982) Least Squares Quantization in PCM. *IEEE Transactions on Information Theory*, 28, 129–137.
[93] Phillips, S.J. (2002) Acceleration of k-means and Related Clustering Algorithms. *Workshop on Algorithm Engineering and Experimentation*, 166–177.
[94] G. Hamerly. Making k-means even Faster. In *Proc. of the 2010 SIAM Int. Conf. on Data Mining*, pp. 130–140, 2010.
[95] Elkan, C. (2003) Using the Triangle Inequality to Accelerate k-means. *ICML*, 3, 147–153.
[96] Bradley, P.S., Mangasarian, O.L. and Street, W.N. (1997) Clustering via Concave Minimization. *Advances in Neural Information Processing Systems*, 368–374.
[97] MacQueen, J. (1967) Some Methods for Classification and Analysis of Multivariate Observations. *Proc. of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
[98] Kaufman, L. and Rousseeuw, P. (1987) *Statistical Data Analysis Based on the L1-Norm and Related Methods*, pp. 405–416. Elsevier, North Holland, Amsterdam.

Downloaded from https://academic.oup.com/comjnl/advance-article/doi/10.1093/comjnl/bxab179/6425234 by guest on 18 November 2021